Appendix – LSTM Model

This appendix illustrates the overview of data analysis and modelling being applied by LSTM model in this study

LSTM Data Analysis and Modelling Process

Long Short-Term Memory (LSTM) networks, a type of recurrent neural network (RNN)

###Data Analysis

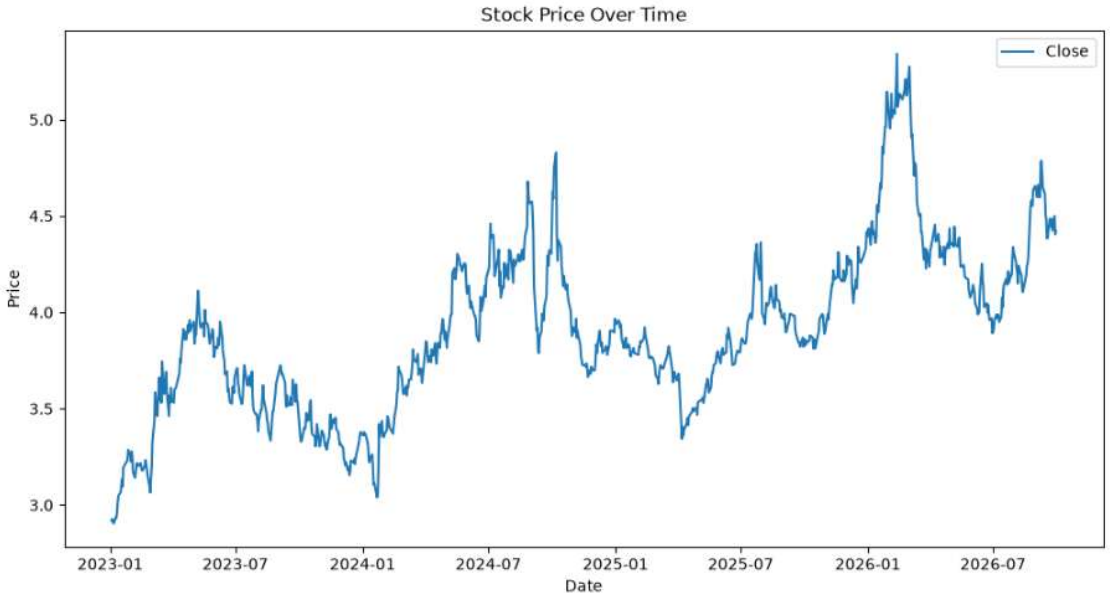
```
# Data source from yfinance, tentatively retreive 3 years stock data for training

company = "0386.HK"
df_HKstock = yf.download(company, start="2023-01-01", end="2026-10-01",
                         auto_adjust=True ).droplevel('Ticker', axis=1)
```

Frist, start investigation on dataset(s) with number of rows, data types, and/or statistic details, etc

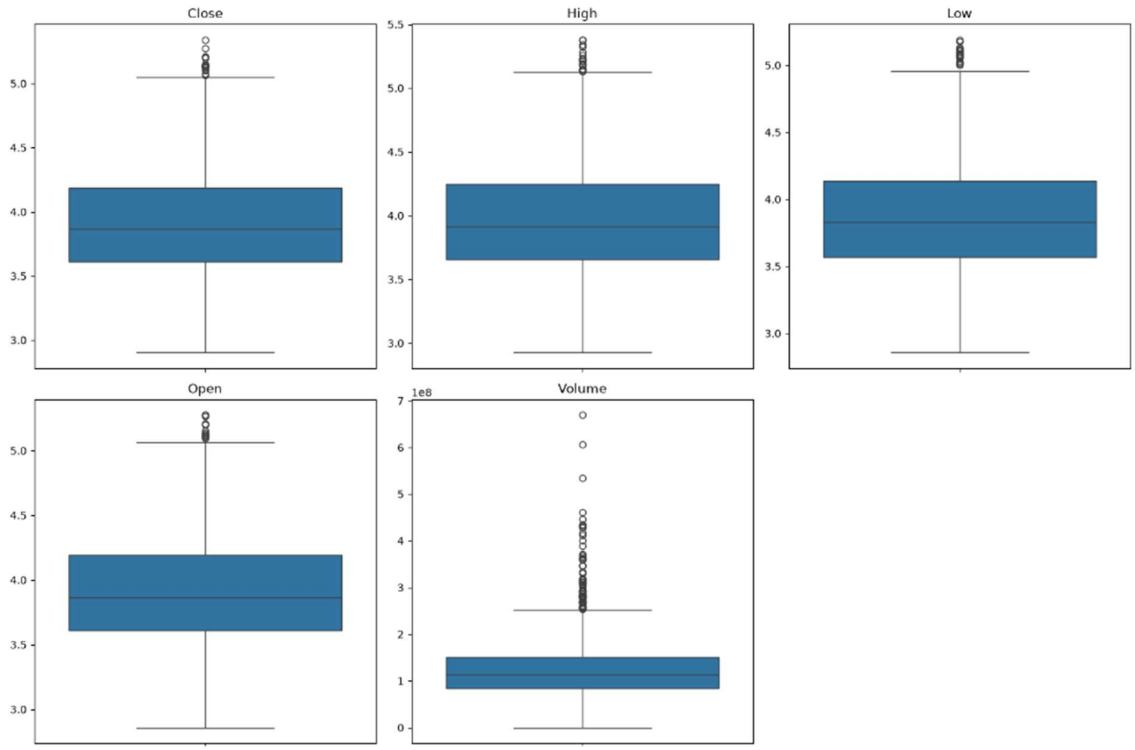
And then build time-series data with Date

Further plot charts to visualize data trend and/or patterns



To study outliners and variations thru boxplot /violinplot

If any box plot has extreme outliers, it may suggest anomalous trading days or incorrect data points.



check null and bfill / ffill if necessary

As modeling required, moving averages and daily returns will be used and handled.

```
# Calculate moving averages and daily returns

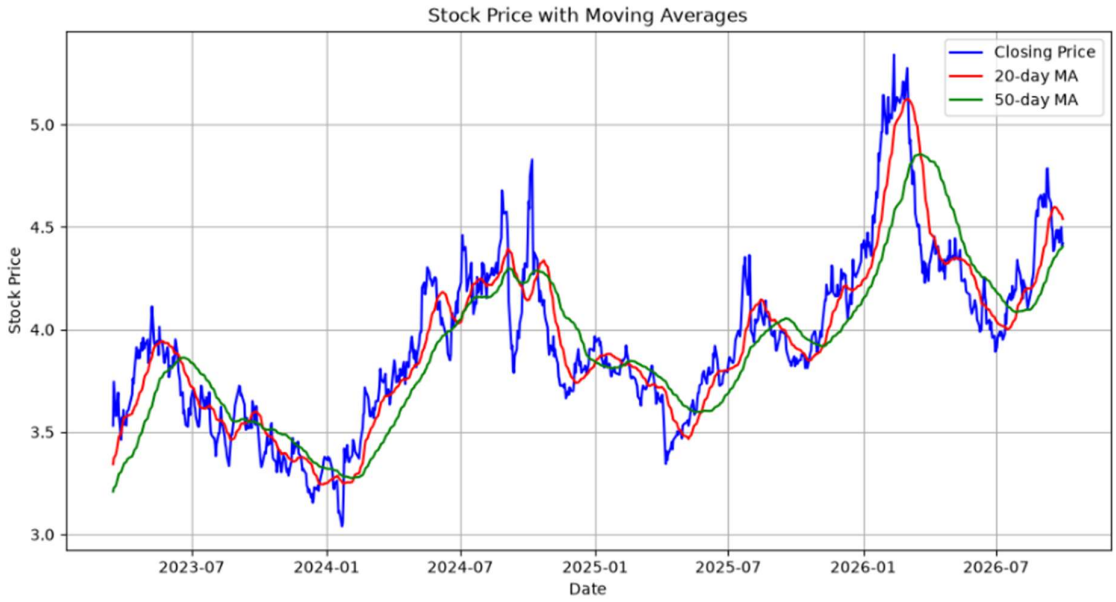
df['20-day MA'] = df['Close'].rolling(window=20).mean()
df['50-day MA'] = df['Close'].rolling(window=50).mean()
df['Daily Return'] = df['Close'].pct_change()
```

```
# To check NaN
df[df.isna().values==True].drop_duplicates()
```

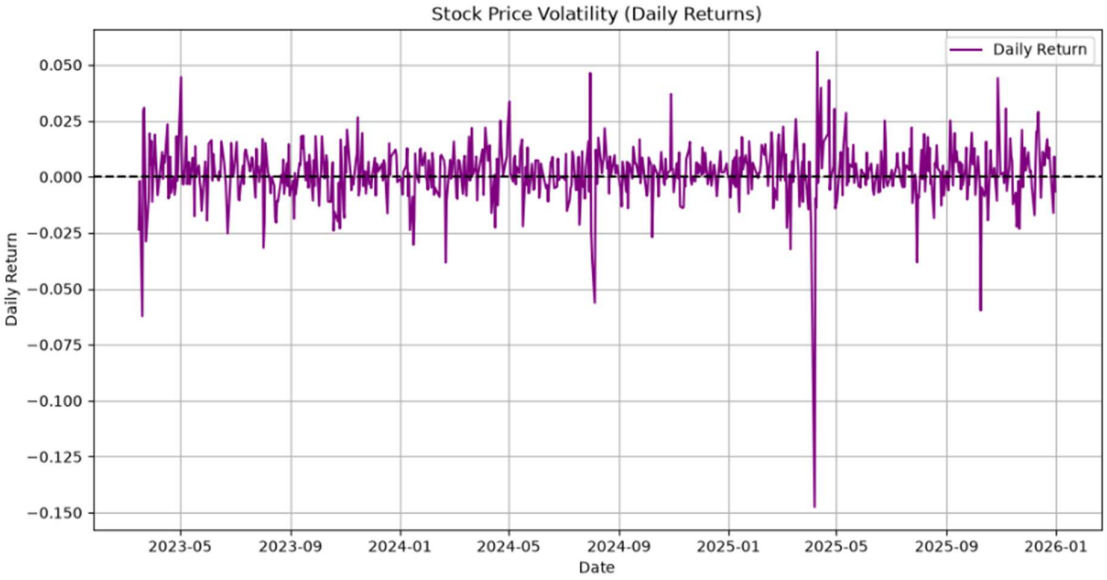
Date	Close	High	Low	Open	Volume	20-day MA	50-day MA	Daily Return
2023-01-03	316.281708	316.831432	301.255591	306.019968	31541576	NaN	NaN	NaN
2023-01-04	330.758087	330.758087	319.763362	320.679579	39409103	NaN	NaN	0.045771
2023-01-05	335.513855	349.405194	331.462239	345.353549	40803927	NaN	NaN	0.014378
2023-01-06	337.057373	346.704121	334.549213	341.109019	27092487	NaN	NaN	0.004600
2023-01-09	349.212280	349.598144	339.179668	342.459557	28323733	NaN	NaN	0.036062
2023-01-10	349.598145	352.878033	344.388906	352.106305	19854116	NaN	NaN	0.001105
2023-01-11	360.595428	365.418802	353.649788	356.736729	35722217	NaN	NaN	0.031457
2023-01-12	351.141632	366.190565	343.810098	365.611754	40547528	NaN	NaN	-0.026217

```
df = df.dropna()   # Drop all NaN
```

20-day MA captures short-term fluctuations, while the 50-day MA provides a smoother long-term trend. Sharp deviations between moving averages could indicate trend reversals.



Observations Large spikes indicate high volatility days, often due to major market events.



Statistics highlight

```
df.describe(include='all').style.format().background_gradient(cmap='Blues')
```

	Close	High	Low	Open	Volume	20-day MA	50-day MA	Daily Return
count	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000
mean	3.941670	3.988927	3.895577	3.940422	131586694.022989	3.930092	3.908141	0.000362
std	0.407925	0.416444	0.397489	0.408014	74553745.462015	0.390258	0.372173	0.016183
min	3.039689	3.105769	3.006649	3.039689	0.000000	3.244124	3.209413	-0.082927
25%	3.669550	3.708491	3.629844	3.670392	84997863.000000	3.637296	3.615765	-0.009109
50%	3.895249	3.941887	3.850783	3.893846	115619236.000000	3.884894	3.881182	0.000000
75%	4.206942	4.262172	4.153887	4.211630	153326906.250000	4.196517	4.157898	0.009535
max	5.341063	5.378809	5.190078	5.275007	669423311.000000	5.124495	4.856026	0.074380

###Modelling

Stock prices fluctuate due to various factors: company earnings, economic trends, geopolitical events, and market sentiment. While no model can predict prices with 100% accuracy, deep learning techniques may discover trends and patterns in historical data.

Traditional models work well for linear time series, but struggle with the long-term dependencies in stock market data.

```
# Configuration

# For ~ 80:20 Split
test_days = 135

# choose the number of days on which to base our predictions
nb_days = 30
#look_back2 = 30

# Number of days into the future we want to predict
n_steps_out = 10

#
epochsMS=15
batch_sizeMS = 32

epochs2=10
batch_size2 = 32

look_back2 = 30
units2 = 32
Dropout2 = 0.2
```

```
# For normailized Data

# from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(df['Close'].values.reshape(-1,1))
```

Remarks: Models learn faster and perform better when features are on the same scale (0,1)

#For LSTM Modelling, the requirement of stationary series is checked through ADF or DFT

```
# ADF Test : p-value < 0.05 for stationary data

#from statsmodels.tsa.stattools import adfuller

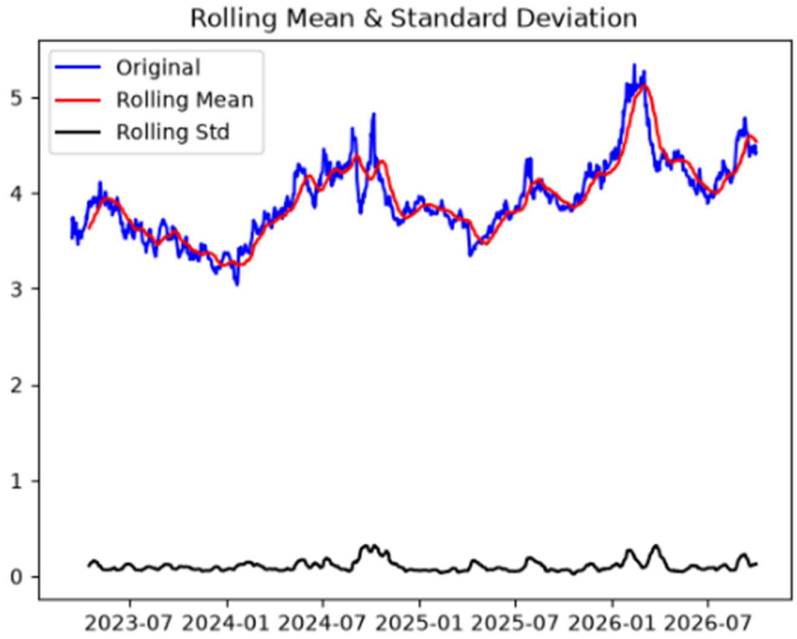
close_series = df['Close']
def check_stationarity(series):
    result = adfuller(series.dropna())  # Drop NaN values if any
    print("ADF Statistic:", result[0])
    print("p-value:", result[1])
    if result[1] < 0.05:
        print("The series is stationary.")
    else:
        print("The series is not stationary.")

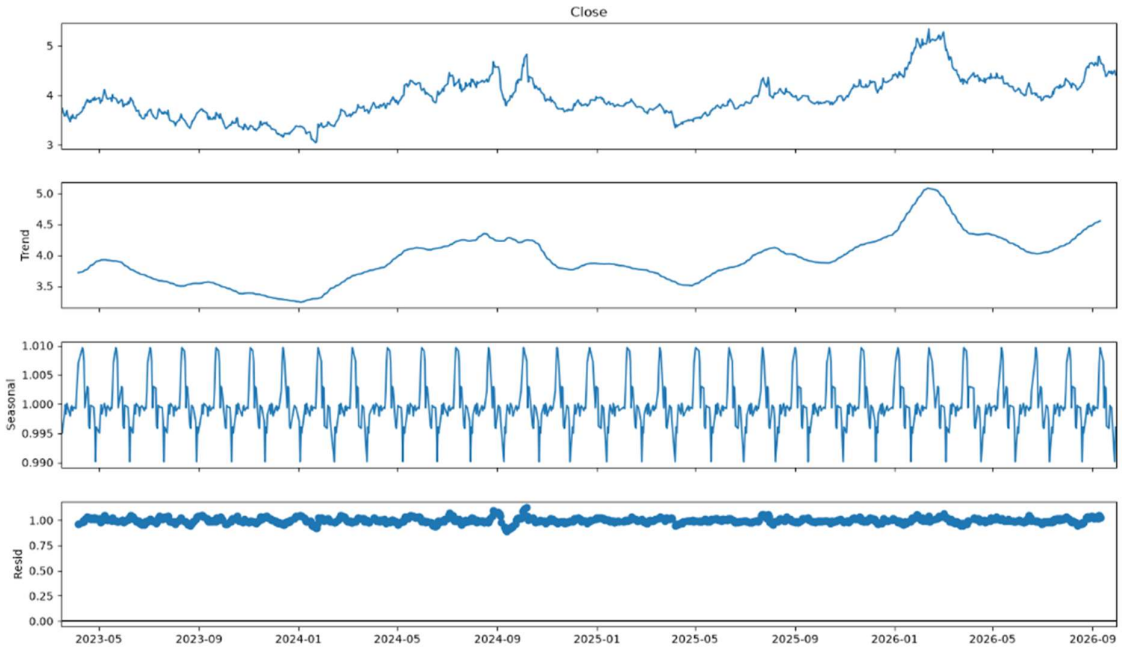
check_stationarity(close_series)
```

```
ADF Statistic: -2.411782424107517
p-value: 0.1384384843994788
The series is not stationary.
```

There are three major time series patterns: trend, seasonality, and cycles. Trend and cycles are usually combined into a single component, with a trend-cycle component, a seasonal component, and a remainder component (containing the rest of the time series).

To make time series stationary, the principle is to estimate trend and seasonality, and to remove those from the time series. Then to apply the forecasting technique, the final step being to convert the forecasted values into the original scale by adding the estimated trend and seasonality.





To apply transformation as

```
#... To apply log transform to reduce the trends effect while square root transform to flatten

df_close_log = df_close.apply(np.log)
df_close_tf = df_close_log.apply(np.sqrt)
```

```
#... classical methold differenciating to handel with trand and seasonality
df_close_shift = df_close_tf - df_close_tf.shift()
```

check p-value

Train : Test split ~(80:20)



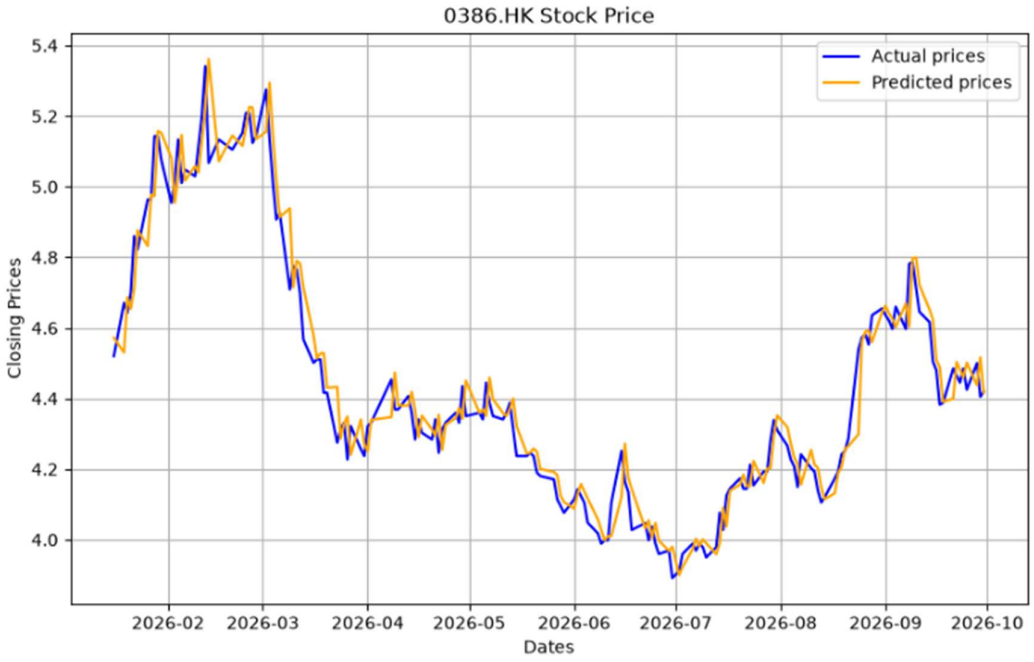
Execute model to view result

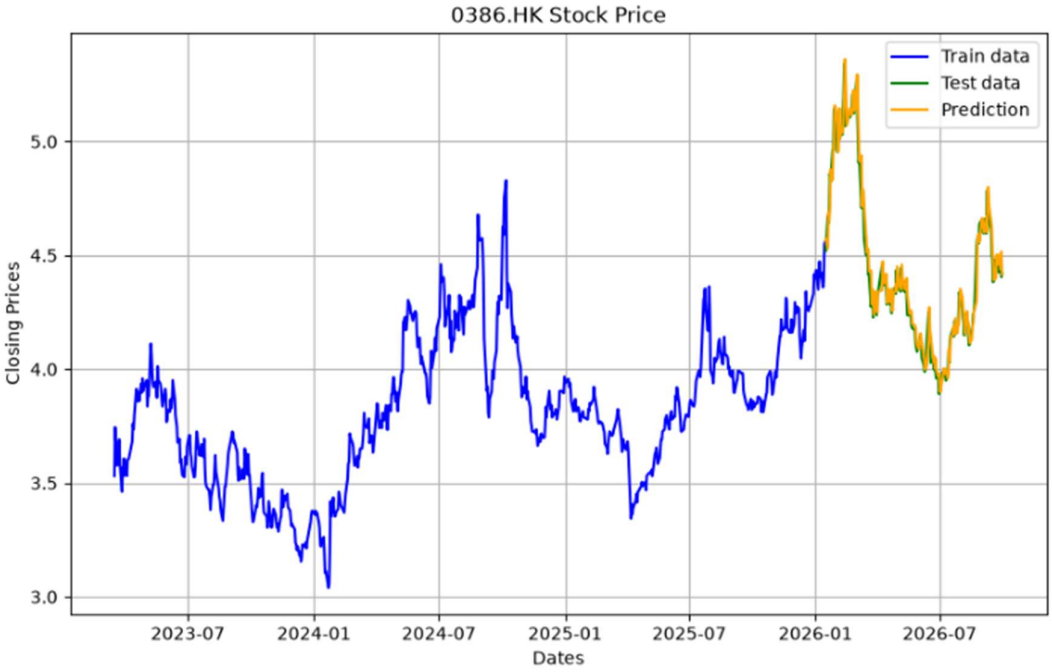
```
#... LSTM Vanilla LSTM
def vanilla_LSTM():
    model = Sequential()
    model.add(LSTM(units=50, input_shape=(nb_days, n_features)))
    model.add(Dense(1))
    return model

#
model = vanilla_LSTM()
model.summary()
model.compile(optimizer='adam',
              loss='mean_squared_error',
              metrics=[tf.keras.metrics.MeanAbsoluteError()])
```

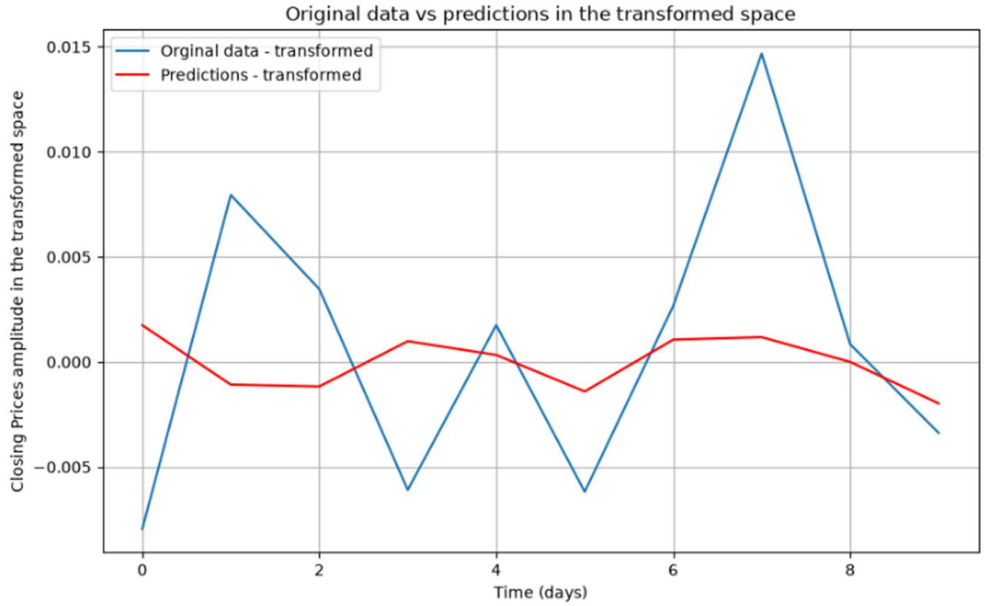
```
#...Train and evaluate model
model.fit(X_train,
          y_train,
          epochs=epochsMS,
          batch_size = batch_sizeMS)
```

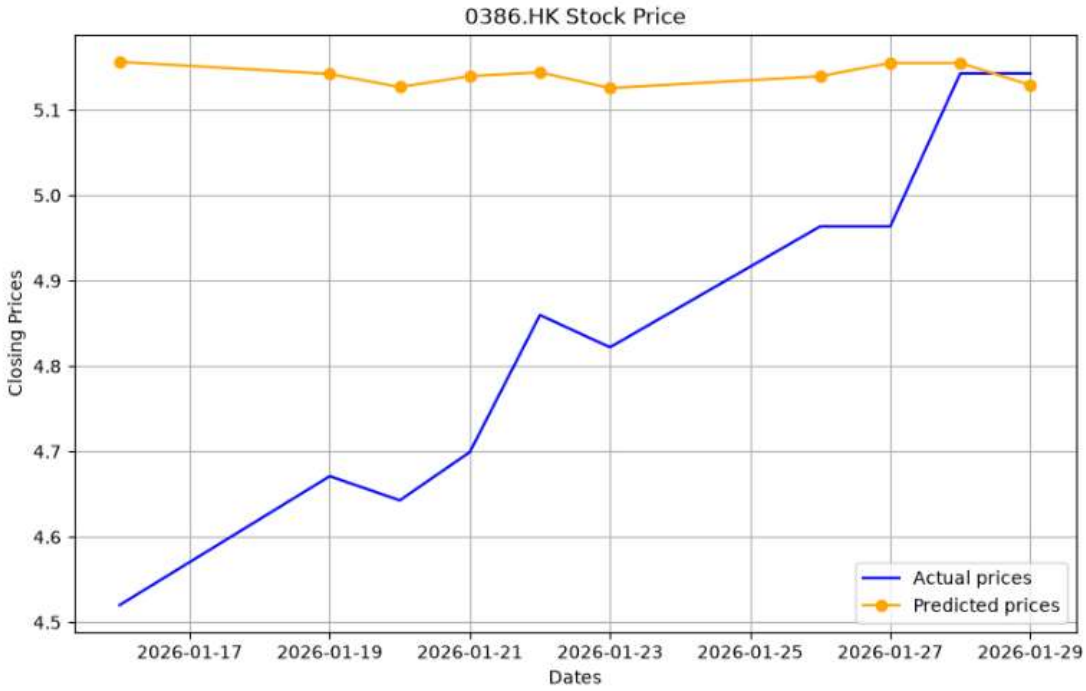
Evaluate and predict with test set, then plot to visualize


0386.HK Stock Price
Actual prices
Predicted prices
Closing Prices
5.4
5.2
5.0
4.8
4.6
4.4
4.2
4.0
2026-02
2026-03
2026-04
2026-05
2026-06
2026-07
2026-08
2026-09
2026-10
Dates


0386.HK Stock Price
Train data
Test data
Prediction
Closing Prices
5.0
4.5
4.0
3.5
3.0
2023-07
2024-01
2024-07
2025-01
2025-07
2026-01
2026-07
Dates

Multi-step LSTM enhance with 10 days to predict





Next, output Metrics for assessment: MAE, RMSE, MAPE

Preliminary Findings:

LSTM captures complex non-linear price dynamics and trend patterns derived from moving-average engineering

Integration of Dropout layers and MinMaxScaler normalization effectively prevents overfitting and stabilizes gradient updates across volatile market regimes.

LSTM Model 2 (version)

LSTM(32 units) → Captures long-term dependencies

Dropout(0.2) → Prevents overfitting

Dense Layers → Produces final prediction

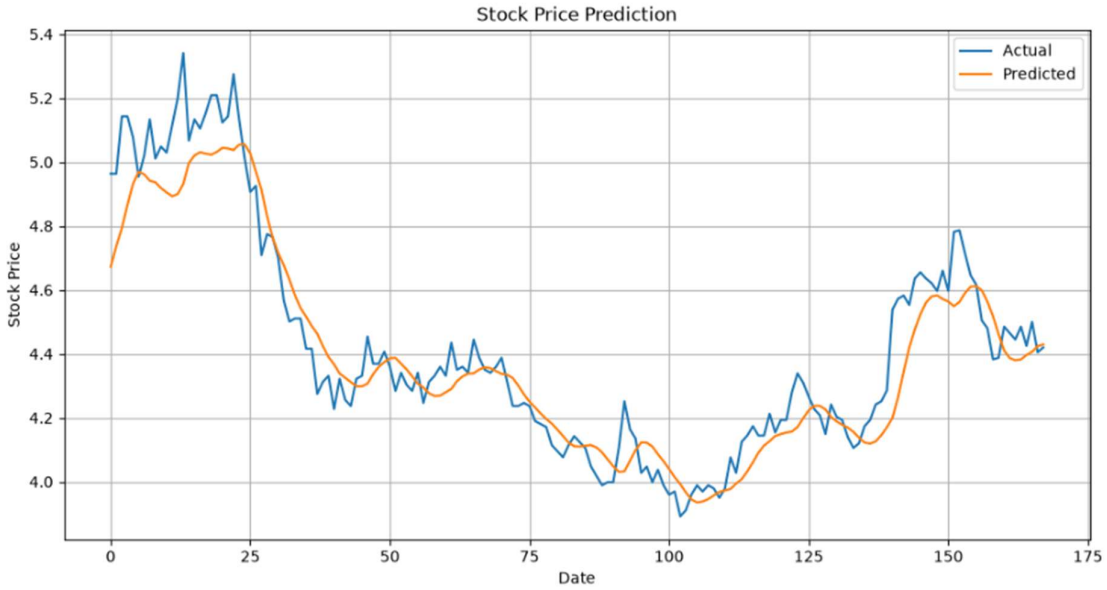
```
#2 LSTM
#from tensorflow.keras.models import Sequential
#from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential([
            LSTM(units=32, return_sequences=True, input_shape=(trainX.shape[1], 1)),
            Dropout(0.2),
            LSTM(units=32, return_sequences=False),
            Dropout(0.2),
            Dense(units=16),
            Dense(units=1)
])

model.compile(optimizer='adam', loss='mean_squared_error')

model.fit(trainX, trainY, epochs=epochs2, batch_size=batch_size2)
```

Model output



Model Tuning with GridSearchCV

```
# 1. Wrap the Keras model into a Scikit-Learn compatible Regressor
# Basic default parameters are set here and will be optimized by the param_grid
keras_regressor = KerasRegressor(
    model=create_vanilla_multistep_LSTM,
    epochs=epochsMS,
    batch_size=batch_sizeMS,
    verbose=0
)

# 2. Define the hyperparameter search grid
# Note: SciKeras requires internal model parameters to be prefixed with "model__"
param_grid = {
    'model__units': [16, 32, 64],
    'model__optimizer': ['adam', 'rmsprop'],
    'batch_size': [16, 32, 64],
    'epochs': [20, 30, 50]
}

# 3. Initialize GridSearchCV
# Using 3-Fold Cross-Validation for hyperparameter grid optimization
grid = GridSearchCV(
    estimator=keras_regressor,
    param_grid=param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1   # Uses all available CPU cores for parallel processing
)

print("Executing GridSearchCV optimization process...")
grid_result = grid.fit(X_train, y_train)

# 4. Display optimization results
print(f"Best Cross-Validation Score (MSE): {-grid_result.best_score_:.6f}")
print(f"Optimal Hyperparameter Configuration: {grid_result.best_params_}")
```

```
Executing GridSearchCV optimization process...
Best Cross-Validation Score (MSE): 0.000051
Optimal Hyperparameter Configuration: {'batch_size': 32, 'epochs': 20, 'model__optimizer': 'adam', 'model__units': 16}
```


```
# 2. Define the hyperparameter search grid
# Note: SciKeras requires internal model parameters to be prefixed with "model__"
param_grid = {
    'model__units': [16, 32, 64],
    'model__optimizer': ['adam', 'rmsprop'],
    'batch_size': [16, 32, 64],
    'epochs': [20, 30, 50]
}
```

```
Executing GridSearchCV optimization process...
Best Cross-Validation Score (MSE): 0.000051
Optimal Hyperparameter Configuration: {'batch_size': 32, 'epochs': 20, 'model__optimizer': 'adam', 'model__units': 16}
```


```
import pickle
with open ('optimal_model.pkl', 'wb') as f:
    pickle.dump(model, f)
```

```
with open('optimal_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)
```

Re-visit with Daily Return

```
import pickle
with open ('model.pk1', 'wb') as f:
    pickle.dump(model, f)
```

```
with open('model.pk1', 'rb') as f:
    loaded_model = pickle.load(f)
```

```
pred_scaled_returns = loaded_model.predict(X_test)
pred_returns = scaler.inverse_transform(pred_scaled_returns).flatten()
actual_returns = df_target.iloc[-test_days:].values.flatten()

r2_returns = r2_score(actual_returns, pred_returns)
r2_prices = r2_score(true_prices, predicted_prices)

print("--- Reloaded Model Summary ---")
print(f"Returns Prediction R² Score: {r2_returns:.4f}")
print(f"Price Reconstruction R² Score: {r2_prices:.4f}")
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
--- Reloaded Model Summary ---
Returns Prediction R² Score: -0.0104
Price Reconstruction R² Score: 0.9705
```

Remarks: scale should be highly considered in comparison

For Automation build

```
import pandas as pd
import numpy as np
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
# 1. Load top 30 stock symbols from stocklisttop30.csv
df_stocklist = pd.read_csv('stocklisttop30.csv')
symbols = df_stocklist['Symbol'].tolist()

results = []

for symbol in symbols:
    try:
        print(f"Processing {symbol}...")
        df = yf.download(symbol, start="2023-01-01", end="2026-01-01", auto_adjust=True, progress=False)
        if isinstance(df.columns, pd.MultiIndex):
            df = df.droplevel('Ticker', axis=1)
        if df.empty or len(df) < 100:
            continue

        df['Close'] = df['Close'].bfill().ffill()
        scaler = MinMaxScaler()
        scaled_close = scaler.fit_transform(df[['Close']])

        def create_dataset(dataset, look_back=30):
            dataX, dataY = [], []
            for i in range(len(dataset) - look_back):
                dataX.append(dataset[i:(i + look_back), 0])
                dataY.append(dataset[i + look_back, 0])
            return np.array(dataX), np.array(dataY)
```

R-squared notes

, R^2 is defined as:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_{actual} - y_{pred})^2}{\sum(y_{actual} - \bar{y})^2}$$

This compares the predictions of your model against a simple horizontal line representing the mean of the actual data.

negative R-squared result as the model performs worse than just guessing the average price every time.